Book 3

The Prompt Library

141 paste-ready prompts, tagged and categorised

Every prompt here can be pasted into Genius as-is. Each one is tagged with how demanding it is, which chat model suits it, and which actions it is expected to call — so you can see what a request is about to do before you run it. They run from single-action starters to expert closed loops that measure their own results.

In this book

- 141 prompts across 17 categories
- Complexity tags from Starter to Expert
- A suggested model for every prompt, and why
- The expected action list for each, cross-referenced to Book 5
- Practical notes: what to expect, what it costs, what to watch for

Every action named in this book is verified at build time to exist in the shipping code.

How to read an entry

Complexity	How much can go wrong. **Starter** is one action with no prerequisites. **Intermediate** chains a few, or needs real parameters. **Advanced** reads state then acts on it. **Expert** runs loops, verifies itself, or builds for a long time unattended.
Model	A suggestion, not a requirement. It reflects what the job actually demands — precise sequencing, long-form explanation, or just speed.
Uses	The actions the prompt is expected to call. Genius may choose differently; this tells you the intended shape and points you at Book 5 for the detail.
Note	What to expect, what it costs, or the trap to avoid.

Complexity tags used in this book:

STARTER INTERMEDIATE ADVANCED EXPERT

Category	Prompts
First Contact & Sanity Checks	8
Reading Your Set	10
Tempo, Transport & Global Feel	12
Track & Mixer Work	9
Writing MIDI — Drums	11
Writing MIDI — Chords, Bass & Melody	10
Clip Shaping & Launch Behaviour	8
Automation & Movement	8
Devices, Racks & Sound Design	11
Arrangement & Song Building	9
Sample Hunting	4
Audio Analysis & Mix Feedback	6
Music Theory & Learning	7
Project Archaeology	6
Diagnostics & Recovery	6
Advanced Multi-Action Chains	10
Expert — Closed Loops & Self-Verification	6
Total	**141**

First Contact & Sanity Checks

Run these before anything else, especially after restarting Live or reinstalling the bridge device. If these don't work, nothing else will.

STARTER ANY

“Connect to Ableton and tell me what you can see.”

Uses: `live_connect`

Note: The canonical first prompt. Confirms the bridge is alive and reports the Set name, tempo and track count.

STARTER ANY

“Are you connected to Live right now? If not, say so plainly instead of guessing.”

Uses: `live_connect`

Note: Explicitly asking for honesty here is worth it — it discourages the model from narrating actions it never performed.

STARTER GEMINI

“What's my current tempo, time signature and global scale?”

Uses: `live_connect` · `live_read_set`

Note: A single cheap fact lookup — exactly the kind of job a fast model handles fine.

STARTER ANY

“How hard is Live working right now? Check the CPU.”

Uses: `live_read_cpu`

Note: Reports Live's own average and peak audio engine load.

STARTER CLAUDE

“Something's frozen and my last request timed out. Check whether a dialog box is blocking Live.”

Uses: `live_read_dialog`

Note: The single most useful recovery prompt in this book. A stuck modal silently blocks every other action.

STARTER ANY

“Post 'Genius is connected' into Live's status bar so I can confirm you're really talking to my session.”

Uses: `live_show_message`

Note: Proof-of-life you can see with your own eyes, in Live's own window.

STARTER ANY

“List every main view Live has, and tell me which ones are visible right now.”

Uses: `live_view_control`

Note: Should return six views. If you only get one, your bridge device predates v1.0.40.

STARTER GEMINI

“Am I synced to anything over Ableton Link? How many peers?”

Uses: `live_read_link_status`

Note: Worth checking whenever the tempo refuses to change — Link may be driving it.

Reading Your Set

Getting the current state of the project into Genius's head. Almost every good advanced prompt starts with one of these.

STARTER CLAUDE

"Give me a full inventory of this Set: every track, what's on it, and what clips exist."

Uses: `live_read_set`

Note: The wide-angle read. On a big Set this is a large payload, so expect it to take a moment.

INTERMEDIATE ANY

"Just read tracks and mixer values — I don't need routings or locators."

Uses: `live_read_set`

Note: Narrowing the include list is the main way to keep big reads cheap.

STARTER ANY

"What am I currently clicked on in Live?"

Uses: `live_get_selected`

Note: Lets you say 'this clip' in later prompts instead of counting track numbers.

INTERMEDIATE CLAUDE

"Tell me everything about the track I've currently got selected."

Uses: `live_get_selected` · `live_read_track`

Note: A two-step chain: find out what's selected, then read it properly.

STARTER ANY

"Read track 3 and tell me what devices are on it and in what order."

Uses: `live_read_track`

Note: Device order is the thing people most often get wrong when describing their chain from memory.

INTERMEDIATE CLAUDE

"List all my scenes and tell me which ones are actually full versus mostly empty."

Uses: `live_read_set` · `live_read_scene`

Note: Requires judgement about what 'mostly empty' means, so a reasoning-heavy model helps.

STARTER GEMINI

"How long is my arrangement right now, in bars?"

Uses: `live_read_arrangement_length`

Note: Returns beats; the model converts to bars using your time signature.

INTERMEDIATE ANY

"Show me every clip sitting on track 2's arrangement timeline with its start position."

Uses: `live_read_arrangement_clips`

Note: You need this before moving anything — arrangement clips are addressed by their index in this list.

INTERMEDIATE CLAUDE

"Read the notes in the clip on track 1, slot 1 and describe the rhythm to me in words."

Uses: `live_read_clip`

Note: Tests whether the model can turn a raw note list back into a musical description.

INTERMEDIATE CLAUDE

"Take a full snapshot of this session and hold onto it. I'm about to make a mess."

Uses: `live_session_snapshot`

Note: The 'before' half of a diff. Worth doing before any recipe or batch operation.

Tempo, Transport & Global Feel

Set-wide controls: speed, swing, groove, looping and recording modes.

STARTER ANY

"Set the tempo to 128 BPM."

Uses: `live_update_set`

Note: The simplest possible write action. A good first test that Genius can change things, not just read them.

INTERMEDIATE ANY

"Set the tempo to 140, the time signature to 6/8, and the global scale to F# minor."

Uses: `live_update_set`

Note: All three land in one call. Setting the global scale makes Live 12's scale-aware editing agree with what Genius writes later.

INTERMEDIATE CLAUDE

"Start playback, let it run, then stop after you've told me what you hear in the arrangement."

Uses: `live_playback` · `live_read_arrangement_clips`

Note: Genius cannot literally hear playback — it will describe what's on the timeline. Book 4 covers this distinction.

INTERMEDIATE ANY

"Put the arrangement loop around bars 17 to 25 and turn looping on."

Uses: `live_set_loop_region`

Note: Beats, not bars, under the hood — the model does the conversion. Verified working live in v1.0.40.

INTERMEDIATE ANY

"Give the whole Set 30% swing, then bump the global groove amount to half."

Uses: `live_set_swing` · `live_set_groove_amount`

Note: Two different systems: swing affects quantized recording/playback, groove scales grooves already attached to clips.

INTERMEDIATE ANY

"Turn on punch-in and punch-out so I only record inside the loop."

Uses: `live_set_punch` · `live_set_loop_region`

Note: Punch without a loop region set does nothing useful, so these two belong together.

STARTER ANY

“My automation stopped moving after I grabbed a knob. Fix it.”

Uses: `live_re_enable_automation`

Note: Almost always the correct answer to that exact complaint.

STARTER ANY

“The arrangement won't play what I can see on the timeline. Sort it out.”

Uses: `live_back_to_arranger`

Note: Session clips are overriding the Arrangement; this is Live's own Back to Arrangement button.

STARTER ANY

“Stop every clip that's playing right now.”

Uses: `live_stop_all_clips`

Note: Respects launch quantization, so it may wait until the next bar line.

INTERMEDIATE ANY

“Unarm every track, unsolo everything, and unmute everything. Clean slate.”

Uses: `live_batch_tracks`

Note: One call, but it touches every track — and therefore creates many undo steps.

STARTER ANY

“Move the playhead to bar 33 but don't start playing.”

Uses: `live_jump_to_time`

Note: Navigation without commitment.

ADVANCED CLAUDE

“Assign my drum tracks to crossfader side A and my synths to side B, then put the crossfader in the middle.”

Uses: `live_read_set` · `live_update_track` · `live_set_crossfader`

Note: Requires the model to first work out which tracks are drums and which are synths from their names and devices.

Track & Mixer Work

Building and shaping the channel strip.

INTERMEDIATE ANY

“Make me four MIDI tracks called Kick, Snare, Hats and Bass, each a different colour.”

Uses: `live_create_track` · `live_update_track`

Note: A classic starter-session prompt. Colour-coding from the start pays off for the rest of the project.

ADVANCED CLAUDE

“Add two return tracks, one for reverb and one for delay, and load the right device on each.”

Uses: `live_create_track` · `live_create_device` · `live_update_track`

Note: Native devices only — see Book 4 on third-party plugin loading.

STARTER ANY

“Rename track 5 to 'Lead' and make it orange.”

Uses: `live_update_track`

Note: Both changes go in a single call.

INTERMEDIATE ANY

“Set track 2's volume to -6 dB, pan it 20% left, and send 15% to return A.”

Uses: `live_update_track`

Note: live_update_track takes twenty-one optional settings, so a whole channel strip fits in one action.

INTERMEDIATE ANY

“Duplicate track 3 and rename the copy 'Lead Double'.”

Uses: `live_duplicate` · `live_update_track`

Note: Duplicating copies the devices too, which on a plugin-heavy track costs real CPU.

STARTER ANY

“Solo track 4 exclusively — I want to hear only that.”

Uses: `live_batch_tracks`

Note: Exclusive solo unsolos everything else in the same call.

STARTER ANY

“Delete track 7. I'm sure.”

Uses: `live_delete`

Note: Destructive but undoable. Genius should confirm the track name back to you before committing.

ADVANCED CLAUDE

“Read every track's volume and tell me if anything is obviously too loud relative to the others.”

Uses: `live_read_set`

Note: Reasoning over the mixer state. This is structural judgement, not listening — see the analysis section for real measurement.

INTERMEDIATE ANY

“Collapse all my group tracks so I can see the session properly.”

Uses: `live_read_set` · `live_set_fold_state`

Note: Needs to find the groups first, then fold each one.

Writing MIDI — Drums

Getting beats into Live. These use deterministic generators, so the same prompt produces the same pattern every time.

STARTER ANY

“Write a basic four-on-the-floor kick pattern into track 1, slot 1.”

Uses: `live_write_notes`

Note: The simplest generative write. A good proof that note-writing works end to end.

INTERMEDIATE ANY

"Put a Euclidean rhythm with 5 pulses across 16 steps on track 2, slot 1."

Uses: `live_generate_euclidean`

Note: Bjorklund's algorithm — the maths behind a huge amount of techno and Afro-Cuban rhythm.

INTERMEDIATE CLAUDE

"Give me three Euclidean variations on track 2: 3-in-16, 5-in-16 and 7-in-16, in slots 1, 2 and 3."

Uses: `live_generate_euclidean`

Note: Three calls. Because the generator is deterministic, these are reproducible variations rather than random ones.

INTERMEDIATE ANY

"Here's a drum grid. Write it to track 1, slot 2.
Kick |x...x...x...x...|
Snare |....x.......x...|
Hat |x.x.x.x.x.x.x.x.|"

Uses: `live_write_drum_grid`

Note: The most legible way to specify a beat, because you can literally see it in the prompt.

ADVANCED CLAUDE

"Write a half-time trap beat with rolling hats and heavy velocity variation into track 1, slot 3."

Uses: `live_create_drum_pattern`

Note: Use the pattern writer rather than the plain grid when dynamics matter as much as placement.

ADVANCED CLAUDE

"Build a 3-against-4 polyrhythm between the kick and the hats on track 1."

Uses: `live_create_drum_pattern`

Note: Polyrhythm support is what separates live_create_drum_pattern from live_write_drum_grid.

INTERMEDIATE ANY

"Show me what's on my drum rack — which pads are loaded and what they're called."

Uses: `live_read_drum_pads`

Note: Needed before writing a beat, so Genius writes to the notes your kit actually uses.

ADVANCED CLAUDE

"Read my drum rack, then write a beat that only uses pads that actually have samples loaded."

Uses: `live_read_drum_pads` · `live_write_drum_grid`

Note: The read-then-act pattern. Prevents the classic failure of writing notes to empty pads.

INTERMEDIATE ANY

"Rename drum pad C1 to 'Kick - Plymouth' and solo it."

Uses: `live_update_drum_pad`

Note: Pads are addressed by the MIDI note that triggers them, not their position on screen.

ADVANCED CLAUDE

"Take the beat in track 1 slot 1, humanize the timing slightly, and put the result in slot 4 so I keep the original."

Uses: `live_transform_clip`

Note: Writing to a different slot is what makes this non-destructive. Say so explicitly or you may lose the original.

INTERMEDIATE ANY

"Bake a 16th-note groove into the clip on track 1 slot 1 at 60% strength."

Uses: `live_commit_groove`

Note: Destructive to the original timing — it rewrites the actual note positions.

Writing MIDI — Chords, Bass & Melody

Harmony and melodic material, with real voice leading rather than stacked triads.

INTERMEDIATE ANY

"Write a i - VI - III - VII progression in A minor into track 3, slot 1."

Uses: `live_create_chord_progression`

Note: Roman numerals mean you can move the same progression to any key by changing one word.

ADVANCED GROK

"Give me a jazz ii-V-I in F major with rootless voicings on track 3, slot 2."

Uses: `live_generate_chords`

Note: Rootless voicings assume a bass instrument is covering the root elsewhere.

ADVANCED GROK

"Write those same chords again but with drop-2 voicings so they sit wider."

Uses: `live_generate_chords`

Note: The voice-leading logic is what stops generated chords sounding like a beginner's MIDI file.

ADVANCED CLAUDE

"Build a walking bassline under the progression in track 3, and put it on track 4."

Uses: `live_read_clip` · `live_generate_walking_bass`

Note: Has to read the chords first — a walking bass walks between changes, so it needs to know the changes.

STARTER ANY

"What key is the clip on track 3 in?"

Uses: `live_detect_key`

Note: Krumhansl-Schmuckler, a published deterministic algorithm — not the model guessing.

ADVANCED CLAUDE

"Detect the key of my lead clip, then write a counter-melody in that key on track 5."

Uses: `live_detect_key` · `live_write_notes`

Note: Detection first means the melody is actually in the right key rather than assumed to be.

INTERMEDIATE ANY

"Take my melody in track 5 slot 1 and write its retrograde into slot 2."

Uses: `live_transform_clip`

Note: Retrograde means backwards. A classical technique that still works beautifully on modern material.

ADVANCED CLAUDE

"Invert the melody in track 5 and transpose it up a fifth, into a new slot."

Uses: `live_transform_clip`

Note: Two transformations. Ask for a new slot or you'll overwrite the source.

ADVANCED CLAUDE

"Double the loop length of track 3 slot 1 and then vary the second half so it isn't a straight repeat."

Uses: `live_duplicate_loop` · `live_read_clip` · `live_write_notes`

Note: The bread-and-butter move for turning a 1-bar loop into a real 2-bar phrase.

INTERMEDIATE ANY

"Remove every note below C2 in track 4 slot 1 — the bass is muddying things up."

Uses: `live_remove_notes`

Note: Surgical: it only touches the pitch window you name.

Clip Shaping & Launch Behaviour

Making clips loop, trim and trigger the way you want — the difference between a static Set and a performable one.

INTERMEDIATE ANY

"Set the clip on track 2 slot 1 to loop between beats 4 and 8 only."

Uses: `live_set_clip_markers`

Note: Trims what plays without editing the underlying notes or audio.

ADVANCED CLAUDE

"Make every clip in scene 1 launch with 1-bar quantization in toggle mode."

Uses: `live_read_scene` · `live_set_clip_launch_mode`

Note: Reads the scene to find the clips, then sets each one. The core of building a performable Session set.

INTERMEDIATE ANY

"Set the clip on track 1 slot 1 to gate mode so it only plays while I hold the pad."

Uses: `live_set_clip_launch_mode`

Note: Gate, trigger, toggle and repeat behave completely differently under a controller.

ADVANCED CLAUDE

"Rename every clip in track 2 to describe what it actually is, based on its notes."

Uses: `live_read_track` · `live_read_clip` · `live_edit_clip`

Note: Needs to read each clip and form a musical judgement about it before naming.

ADVANCED CLAUDE

"Colour-code all my clips by type: drums red, bass blue, harmony green, leads yellow."

Uses: `live_read_set` · `live_update_clip`

Note: The model classifies from track names and content, then writes a colour per clip.

INTERMEDIATE ANY

"Show me the warp markers on the audio clip in track 6 slot 1."

Uses: `live_manage_warp_markers`

Note: Warp markers are how Live stretches audio to the grid. Deleting the wrong one badly smears the sound.

ADVANCED CLAUDE

"This vocal sample is drifting out of time. Add a warp marker at the start of each bar."

Uses: `live_read_clip` · `live_manage_warp_markers`

Note: Has to read the clip length first to know where the bar lines fall.

INTERMEDIATE ANY

"Attach a groove from the groove pool to the clip on track 1 slot 1."

Uses: `live_groove_pool`

Note: Non-destructive, and globally scaled by live_set_groove_amount. Baking it in permanently is live_commit_groove.

Automation & Movement

Making things move over time, procedurally — the fastest way to stop a loop sounding static.

ADVANCED CLAUDE

"Write a filter sweep automation on track 2's Auto Filter that opens across the clip."

Uses: `live_read_device` · `live_set_clip_automation`

Note: Reads the device first to find the right parameter name, then draws the envelope.

ADVANCED CLAUDE

"Put a slow sine LFO on the reverb Dry/Wet in track 3, one full cycle per clip."

Uses: `live_add_shaped_automation`

Note: An LFO-style sweep without an LFO device. Computed locally, written in one go.

INTERMEDIATE ANY

"Add a 4-cycle triangle wave on track 1's volume for a tremolo effect."

Uses: `live_add_shaped_automation`

Note: Cycles controls how many times the shape repeats across the clip length.

ADVANCED CLAUDE

"Draw a random-stepped automation curve on the filter cutoff with 64 points for a glitchy feel."

Uses: `live_add_shaped_automation`

Note: More points means more resolution but a bigger message across the bridge.

STARTER ANY

"Does the clip on track 2 slot 1 have any automation on it already?"

Uses: `live_get_clip_automation`

Note: A cheap pre-flight check before you overwrite something you drew by hand.

INTERMEDIATE ANY

"Clear all the envelopes off track 2 slot 1 — I want to start the automation again."

Uses: `live_clear_clip_envelopes`

Note: Broad and destructive: it doesn't distinguish envelopes Genius wrote from ones you drew.

ADVANCED CLAUDE

"Check whether track 4 slot 1 has automation, and only clear it if it does. Tell me first."

Uses: `live_get_clip_automation` · `live_clear_clip_envelopes`

Note: Conditional execution with a confirmation step — a good habit for destructive actions.

STARTER ANY

"Turn on session automation recording so my knob moves get captured."

Uses: `live_set_automation_record`

Note: Live doesn't reliably report this property back, so the tool trusts the write rather than failing a bogus check.

Devices, Racks & Sound Design

Working with instruments and effects, including the parameter-safety tools added in v1.0.39.

INTERMEDIATE ANY

"What devices are on track 2, and what are their key parameters set to?"

Uses: `live_read_device`

Note: On a plugin with hundreds of parameters this is a large payload — narrow it if you can.

INTERMEDIATE ANY

"Put an EQ Eight and then a Compressor on track 3, in that order."

Uses: `live_create_device`

Note: Native Live devices only. Order matters enormously to the resulting sound.

INTERMEDIATE ANY

"Move the reverb on track 2 so it sits after the compressor instead of before it."

Uses: `live_read_track` · `live_move_device`

Note: Reads the chain to find the current positions, then reorders.

INTERMEDIATE CLAUDE

"Before you change the filter cutoff on track 2, tell me what range it actually accepts."

Uses: `live_read_param_range`

Note: Added in v1.0.39 precisely so Genius stops guessing at legal values. Ask for this whenever a parameter is unfamiliar.

ADVANCED CLAUDE

“Check the range on the Warmth parameter, then set it to 60% of its maximum.”

Uses: `live_read_param_range` · `live_update_device`

Note: The correct pattern for any unfamiliar parameter: read the range, compute the value, then write it.

ADVANCED CLAUDE

“That parameter won't take the value I asked for. Write it raw instead.”

Uses: `live_read_param_range` · `live_set_param_raw`

Note: The honest escape hatch for devices with broken display formatters. Always pair it with a range read.

INTERMEDIATE ANY

“Is the device on track 4 a native Live device or a plugin? Tell me everything about it.”

Uses: `live_inspect_plugin`

Note: Reach for this when Genius claims it can't find a parameter you can clearly see in the GUI.

INTERMEDIATE ANY

“List the macros on the rack in track 1 and tell me what each one does.”

Uses: `live_read_macros`

Note: Read before writing, so you know what Macro 3 is on this particular rack.

STARTER ANY

“Set the 'Brightness' macro on track 1 to 75%.”

Uses: `live_update_macro`

Note: Addressing by name survives someone reordering the macros; addressing by number doesn't.

ADVANCED CLAUDE

“Find every Dry/Wet in the rack on track 2 and bind them all to Macro 1 as one intensity knob.”

Uses: `live_map_rack_magnitude`

Note: Works, but the macro won't get renamed — that half relies on properties absent from real racks. See Book 4.

ADVANCED CLAUDE

“Load a punchy kick sample onto the C1 pad of my drum rack.”

Uses: `live_find_samples` · `live_update_device`

Note: Has to find a real file on disk first, then load it to the pad.

Arrangement & Song Building

Turning loops into an actual track with a beginning, middle and end.

INTERMEDIATE ANY

“Copy the clip in track 1 slot 1 to the arrangement at bar 1, and again at bar 17.”

Uses: `live_duplicate_to_arrangement`

Note: The instant, non-real-time way to assemble an arrangement out of Session ideas.

EXPERT CLAUDE

“Build me a 64-bar arrangement from my session clips: 8-bar intro, 16-bar verse, 16-bar chorus, 8-bar breakdown, 16-bar outro.”

Uses: `live_read_set` · `live_duplicate_to_arrangement` · `live_read_arrangement_length`

Note: One of the highest-value prompts in this book. Take a snapshot first — it makes a lot of changes.

ADVANCED CLAUDE

“Record scene 2 into the arrangement starting at bar 33.”

Uses: `live_record_to_arrangement`

Note: Real time: recording 16 bars genuinely takes 16 bars of wall-clock time.

INTERMEDIATE ANY

“Move the clip at index 3 on track 2 to start at bar 25 instead.”

Uses: `live_read_arrangement_clips` · `live_move_arrangement_clip`

Note: Needs the read first, because arrangement clips are addressed by index.

ADVANCED CLAUDE

“Colour all the chorus clips in my arrangement orange so I can find them.”

Uses: `live_read_arrangement_clips` · `live_update_arrangement_clip`

Note: The cheapest navigation upgrade there is on a long timeline.

INTERMEDIATE ANY

“Add locators at bar 1, 17, 33 and 49 called Intro, Verse, Chorus and Outro.”

Uses: `live_update_set`

Note: Locators then make live_jump_cue useful for navigating the song.

ADVANCED CLAUDE

“Read the MIDI on the arrangement clip at index 2 on track 3 and tell me if it matches the session version.”

Uses: `live_read_arrangement_notes` · `live_read_clip`

Note: Comparing timeline material against its Session source is a common consistency check.

STARTER ANY

“I just played something good but wasn't recording. Rescue it.”

Uses: `live_capture_midi`

Note: Pulls Live's retrospective capture buffer into a new clip. It can't invent a performance that Live didn't buffer.

STARTER ANY

“Create a new scene that captures everything currently playing.”

Uses: `live_create_scene`

Note: The fastest way to freeze a good improvised combination before you lose it.

Sample Hunting

Finding sounds on disk and getting them into Live.

STARTER ANY

“Find me some punchy kick samples.”

Uses: `live_find_samples`

Note: Entirely local, no Ableton needed. Turns a vague description into real file paths.

INTERMEDIATE ANY

“Search my sample library for vinyl-style textures and atmospheric noise.”

Uses: `live_library`

Note: Searches tags and categories too, not just filenames.

ADVANCED CLAUDE

“Find a breakbeat sample, load it into track 6 slot 1, and warp it to my project tempo.”

Uses: `live_find_samples` · `live_create_clip` · `live_update_clip`

Note: Three steps: find the file, create an audio clip from it, then set the warp mode.

ADVANCED CLAUDE

“Find three different snare samples and put them on pads D1, D#1 and E1.”

Uses: `live_find_samples` · `live_update_device`

Note: Pads are addressed by MIDI note, so naming the notes directly is the reliable way to ask.

Audio Analysis & Mix Feedback

The only actions that judge how your music actually sounds. Everything else in this book reasons about structure; these measure reality.

INTERMEDIATE CLAUDE

“Record 15 seconds of my master output and tell me the LUFS.”

Uses: `live_record_and_analyze`

Note: Real time — a 15-second capture takes 15 seconds. This is the only path from Live's audio into real measurement.

INTERMEDIATE ANY

“Analyse this file and tell me if it's loud enough for streaming: C:/renders/mix_v3.wav”

Uses: `live_analyze_audio`

Note: Pure local DSP — Ableton doesn't even need to be open. Returns LUFS, true peak, RMS, crest factor and a spectrum.

ADVANCED CLAUDE

“Compare mix_v2.wav and mix_v3.wav and tell me what actually changed.”

Uses: `live_compare_audio`

Note: The objective answer to 'did that tweak actually help?' rather than a vibes-based one.

ADVANCED CLAUDE

“Audit my mix for problems — low-end clashes, clipping, anything missing.”

Uses: `live_analyze_mix`

Note: Structural and heuristic: it reasons about your Set's layout, it does not listen. Pair with a recording for the full picture.

STARTER ANY

"Check the output meter on the master and tell me if I'm clipping."

Uses: `live_read_meter`

Note: A single instantaneous sample, not a running measurement — don't mistake it for real loudness metering.

EXPERT CLAUDE

"Run a mix audit, then record 20 seconds and measure it, and tell me where the two disagree."

Uses: `live_analyze_mix` · `live_record_and_analyze`

Note: Structural prediction versus measured reality. Where they disagree is usually where the interesting problem is.

Music Theory & Learning

Genius's knowledge base. These don't touch Ableton at all — Live doesn't even need to be running.

STARTER GROK

"Explain the difference between Dorian and Aeolian, and when I'd reach for each."

Uses: `library_db_search`

Note: Hits the structured Music/Audio database, which is always the first place Genius looks.

INTERMEDIATE GROK

"What's the standard signal chain order for mixing a vocal, and why that order?"

Uses: `library_db_search`

Note: The 'why' is what makes this worth asking a long-form model rather than looking it up.

STARTER GROK

"How does sidechain compression actually work? Explain it like I've never used a compressor."

Uses: `library_db_search`

Note: Long-form explanation is exactly what the big-context models are good at.

INTERMEDIATE GEMINI

"What does the Shaper device in Live 12 do? Check the actual manual."

Uses: `library_db_search` · `library_search`

Note: Tries the database first; falls back to your Knowledge folder, so this works best if the Live manual PDF is in it.

INTERMEDIATE ANY

"Look up the manual for my controller in my Knowledge folder and tell me how to set up custom modes."

Uses: `library_search` · `library_read`

Note: Hardware manuals are not bundled. Put the PDF in your Knowledge folder first — this is exactly the fallback case.

ADVANCED GROK

"Teach me what the Live Object Model is and why it matters for what you can do."

Uses: `library_db_search`

Note: Useful background before you start writing prompts that use live_api directly.

ADVANCED CLAUDE

"Explain Euclidean rhythms, then demonstrate by writing three different ones into my Set."

Uses: `library_db_search` · `live_generate_euclidean`

Note: Theory and practice in one prompt — learn the concept, then hear it immediately.

Project Archaeology

Reading saved .als project files straight off disk. Live doesn't need to be running and the project doesn't need to be open.

STARTER ANY

"Summarise this project file without opening it: D:/Projects/old_idea.als"

Uses: `als_summary`

Note: Genuinely useful for triaging a folder full of projects you can't remember.

STARTER ANY

"List the tracks inside that project."

Uses: `als_list_tracks`

Note: A quick look inside before deciding whether it's worth reopening.

INTERMEDIATE ANY

"Pull the MIDI out of track 3 of that old project — I want that bassline back."

Uses: `als_extract_midi`

Note: Recovers a good part without loading the whole project and all its missing plugins.

ADVANCED CLAUDE

"Compare v2 and v3 of this project and tell me what I changed."

Uses: `als_diff`

Note: Effectively version control for Ableton projects.

INTERMEDIATE ANY

"Scan this project and tell me how finished it actually is."

Uses: `als_find_unfinished`

Note: Flags empty tracks, default names and a missing arrangement. Good for picking which half-idea to finish.

ADVANCED CLAUDE

"Go through these five project files and rank them by how close to finished they are."

Uses: `als_summary` · `als_find_unfinished`

Note: Five files, two actions each, then a judgement call. All local — no Ableton required.

Diagnostics & Recovery

When something isn't working. Work down this list in order.

INTERMEDIATE CLAUDE

"Nothing is responding. Walk through the diagnostics: check the connection, check for a dialog, check CPU."

Uses: `live_connect` · `live_read_dialog` · `live_read_cpu`

Note: The standard triage sequence, in the right order. Start here whenever something breaks.

STARTER ANY

"Undo the last thing you did."

Uses: `live_undo`

Note: Live's native undo. One Genius action can be several undo steps, so you may need to repeat it.

INTERMEDIATE ANY

"Undo the last five steps — that whole batch was wrong."

Uses: `live_undo`

Note: Five separate calls. Live's undo history is the safety net for everything in this book.

EXPERT CLAUDE

"Take a snapshot, make the change I asked for, then diff it and show me exactly what moved."

Uses: `live_session_snapshot` · `live_session_diff`

Note: The receipt pattern. Wrap anything risky in this and you get a precise change list rather than a claim.

INTERMEDIATE CLAUDE

"Read the dialog that's blocking Live and press whichever button dismisses it safely."

Uses: `live_read_dialog`

Note: Ask it to read first and tell you what the dialog says before it presses anything.

EXPERT CLAUDE

"Use the raw Live API to read a property the normal actions don't expose."

Uses: `live_api`

Note: The sharpest tool here — it will happily do things no purpose-built action would allow. Know what you're asking for.

Advanced Multi-Action Chains

Where Genius earns its keep: prompts that read the Set, reason about it, and then make coordinated changes across many actions.

EXPERT CLAUDE

"Read my whole Set, work out what genre it's heading towards, then suggest three specific things to add and build whichever I pick."

Uses: `live_read_set` · `live_read_track` · `live_create_track` · `live_write_notes`

Note: Read, reason, propose, wait for a decision, then execute. The conversational pause in the middle is the point.

EXPERT CLAUDE

"Set up a complete techno starting template: 128 BPM, four MIDI tracks named and coloured, a drum rack on track 1, a four-on-the-floor kick, and an off-beat hat pattern."

Uses: `live_update_set` · `live_create_track` · `live_update_track` · `live_create_device` · `live_write_drum_grid`

Note: Roughly a dozen underlying calls from one sentence. This is the flagship demonstration prompt.

EXPERT CLAUDE

"Detect the key of my lead, then build a chord progression that fits it, add a walking bass underneath, and put all three in scene 2."

Uses: `live_detect_key` · `live_create_chord_progression` · `live_generate_walking_bass` · `live_create_scene`

Note: Each step depends on the one before it. If key detection is ambiguous, expect Genius to say so rather than plough on.

EXPERT CLAUDE

"Take my 8-bar loop and turn it into a full arrangement with intro, build, drop, breakdown and outro. Add filter automation on the build."

Uses: `live_read_set` · `live_duplicate_to_arrangement` · `live_add_shaped_automation` · `live_update_set`

Note: The single most ambitious prompt in this book. Snapshot first, and expect to iterate.

EXPERT CLAUDE

"Audit every track: for each one tell me what it is, whether it's mixed sensibly, and what single change would most improve it. Then apply the changes I approve."

Uses: `live_read_set` · `live_read_track` · `live_read_device` · `live_update_track`

Note: Read-heavy, then a batch of approved writes. The approval gate keeps it safe on a real project.

EXPERT CLAUDE

"Build me a dub techno chord stab: make the track, load the chain, write the chords, and put a slow LFO on the delay feedback."

Uses: `live_create_track` · `live_create_device` · `live_generate_chords` · `live_add_shaped_automation`

Note: A complete sound-design build from one sentence. Native devices only.

EXPERT CLAUDE

"Look at what's in scene 1, then create scene 2 as a stripped-back breakdown version of it."

Uses: `live_read_scene` · `live_create_scene` · `live_duplicate` · `live_remove_notes`

Note: Requires a musical judgement about what 'stripped back' means for this specific material.

EXPERT CLAUDE

"Compare every clip in track 1 and delete any that are exact duplicates of each other."

Uses: `live_read_track` · `live_read_clip` · `live_edit_clip`

Note: Read everything, compare locally, then delete. Ask it to list what it will delete before it does it.

EXPERT CLAUDE

"Set up a full sidechain-style pumping effect on track 3 using shaped volume automation locked to the kick pattern in track 1."

Uses: `live_read_clip` · `live_add_shaped_automation`

Note: Reads the kick's actual note positions, then shapes the automation to match them.

ADVANCED CLAUDE

“Go through my whole Set and rename anything still called 'MIDI 1' or 'Audio 2' with something descriptive of what's actually on it.”

Uses: `live_read_set` · `live_read_track` · `live_update_track`

Note: Tedious by hand, trivial to delegate. A good first taste of real batch work.

Expert — Closed Loops & Self-Verification

The most powerful pattern available: make a change, measure the result, and adjust based on what actually happened rather than what was intended.

EXPERT CLAUDE

“Adjust the limiter on my master until a 10-second capture measures between -15 and -13 LUFS. Keep iterating until you get there, and show me each attempt.”

Uses: `live_read_device` · `live_update_device` · `live_record_and_analyze`

Note: The flagship closed loop. Each iteration costs real wall-clock time, so cap the number of attempts.

EXPERT CLAUDE

“Record 10 seconds, analyse the spectrum, and if there's too much energy below 60 Hz, high-pass whatever track is causing it and re-measure.”

Uses: `live_record_and_analyze` · `live_read_set` · `live_create_device` · `live_update_device`

Note: Measure, diagnose, fix, verify. Note it must work out which track is responsible, which may take a couple of passes.

EXPERT CLAUDE

“Snapshot the session, apply the deep house recipe, diff it, and give me a plain-English list of everything it changed.”

Uses: `live_session_snapshot` · `live_apply_recipe` · `live_session_diff`

Note: The safest possible way to run a recipe on a project you care about.

EXPERT CLAUDE

“Before and after: record 10 seconds, make the EQ change I asked for, record another 10 seconds, and tell me measurably what improved.”

Uses: `live_record_audio` · `live_update_device` · `live_compare_audio`

Note: Two real-time captures, so budget the wall-clock time. The payoff is an objective answer.

EXPERT CLAUDE

“Read the parameter range first, then set it, then read it back and confirm the value actually landed where you intended.”

Uses: `live_read_param_range` · `live_update_device` · `live_read_device`

Note: Read-write-verify. The habit that catches silent no-ops on devices with awkward parameter formatting.

EXPERT CLAUDE

“Work through every track that has a rack on it, map its magnitude macro, and report which ones succeeded and which didn't.”

Uses: `live_read_set` · `live_read_device` · `live_map_rack_magnitude`

Note: Expect honest partial success — the macro renaming step is inert, so verify what actually happened rather than trusting the summary.

Writing your own

The prompts that work share a shape. Most of it is just being explicit about things you were assuming.

Start with a read when the request depends on what's there

“Read my drum rack, *then* write a beat using loaded pads only” succeeds where “write me a beat” writes notes to empty pads.

Say where the output goes

Naming a destination slot turns a destructive operation into a non-destructive one.

Give it a measurable target when there is one

“Until it measures between –15 and –13 LUFS” is actionable. “Until it sounds loud enough” is not — and Book 4 explains why it cannot hear it anyway.

Ask for the plan before the execution on anything big

“Tell me what you're going to do first” costs one turn and has saved a great many projects.

Cap the iterations on anything that loops

Each pass of a record-and-measure loop costs real wall-clock time. “Maximum five attempts” keeps it bounded.

Ask for verification, not reassurance

“Diff it and show me what changed” beats “did that work?”, because the first one is checked against Live and the second one is a model's opinion of its own work.